New Advances in Approximate Inference for Logical Credal Networks

David S. Hippocampus*
Department of Computer Science
Cranberry-Lemon University
Pittsburgh, PA 15213
hippo@cs.cranberry-lemon.edu

April 5, 2026

Abstract

Logical Credal Networks (LCNs) are an expressive probabilistic logic that combines propositional logic formulas with imprecise probability bounds to specify sets of probability distributions. While LCNs are highly expressive—allowing cyclic dependencies and few syntactic restrictions—inference remains challenging: existing exact methods scale exponentially with the number of propositions, and the only available approximate scheme relies on a single message-passing algorithm (ARIEL). In this paper, we exploit the chain graph structure underlying LCNs to decompose inference into local credal network problems, and we introduce three new approximate inference algorithms that operate on this decomposition: (1) *Credal Variable Elimination* (CVE) with ε-pruning, an FPTAS that produces outer bounds with controllable accuracy; (2) *Interval Belief Propagation* (IBP), a message-passing scheme that propagates interval-vector messages and produces outer bounds; and (3) *ApproxLP*, a coordinate descent method that produces complementary inner bounds. We evaluate these algorithms on a suite of randomly generated and benchmark LCN instances, demonstrating significant speedups over exact inference while maintaining high-quality probability bounds.

1 Introduction

Combining logical and probabilistic reasoning is a long-standing goal of artificial intelligence. Frameworks such as Markov Logic Networks [**?**], ProbLog [**?**], and Probabilistic Soft Logic [**?**] allow encoding domain knowledge through logical rules augmented with uncertainty. However, most of these formalisms require precise probability values and make strong structural assumptions (e.g., acyclicity).

*Use footnote for providing further information about author (webpage, alternative address)—*not* for acknowledging funding agencies.

Logical Credal Networks (LCNs) [**?**] offer a more general approach: they allow *imprecise* probability specifications in the form of bounds $\alpha \leq P(\phi|\varphi) \leq \beta$ on arbitrary propositional logic formulas, without requiring acyclicity. An LCN defines a *credal set*—a convex set of probability distributions consistent with the given constraints—and inference amounts to computing tight lower and upper bounds on posterior probabilities of query formulas.

Previous work on LCNs has introduced exact inference via constrained optimization [**?**], approximate message-passing via the ARIEL scheme [**?**], and abductive reasoning [**?**]. The factorization properties of LCNs through chain graphs have also been studied [**?**]. However, the exact approach scales exponentially with the number of propositions, and ARIEL is the only available approximate algorithm, limiting the practical applicability of LCNs to small instances.

In this paper, we make the following contributions:

1. We show how the chain graph structure of an LCN can be exploited to decompose the global inference problem into local credal network inference tasks, bridging LCNs and the well-studied credal network formalism.

2. We introduce three new approximate inference algorithms that operate on this decomposition:

   - **Credal Variable Elimination (CVE)** with ε-pruning: a bucket elimination scheme that produces *outer bounds* and constitutes a fully polynomial-time approximation scheme (FPTAS) for bounded treewidth networks.
   - **Interval Belief Propagation (IBP)**: a message-passing algorithm that propagates interval-vector messages on the factor graph and produces *outer bounds*, with native support for multi-valued variables.
   - **ApproxLP**: a coordinate descent algorithm that produces complementary *inner bounds* by iteratively optimizing over extreme points.

3. We provide an experimental evaluation on randomly generated and benchmark LCN instances, comparing accuracy and runtime of all three algorithms against exact inference.

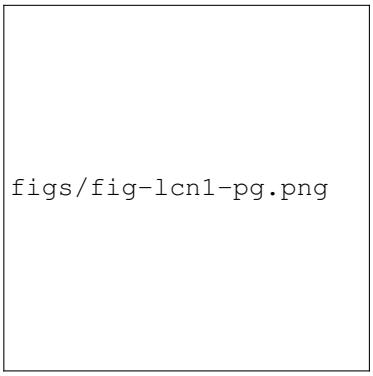
2 Background

2.1 Logical Credal Networks

A Logical Credal Network (LCN) consists of a set of propositions $\mathcal{A}$ and two sets of constraints $\mathcal{T}_U$ and $\mathcal{T}_D$. The set $\mathcal{A}$ is finite with propositions $A_1, \ldots, A_n$. Each proposition A_i is associated with a binary indicator variable X_i: if A_i holds in an interpretation then $X_i = 1$; otherwise, $X_i = 0$. Each constraint in $\mathcal{T}_U$ and in $\mathcal{T}_D$ is of the form:

$$\alpha \leq P(\phi|\varphi) \leq \beta, \tag{1}$$

where ϕ and φ are propositional logic formulas over $\mathcal{A}$. The formula φ can be a tautology $\top$, yielding an unconditional constraint $P(\phi)$. There is no syntactic difference



(a) Primal graph

(b) Structure

Figure 1: Primal graph and structure of the LCN from Example 1.

between constraints in $\mathcal{T}_U$ and $\mathcal{T}_D$; the distinction lies in how they influence the graphical structure of the LCN (as described below).

Primal Graph of the LCN. The semantics of an LCN is given by a translation to a directed graph called the *primal graph*. Each proposition is a node (a *proposition-node*). Each constraint $\alpha \leq P(\phi|\varphi) \leq \beta$ is processed as follows:

1. A node labeled with formula ϕ is added (a *formula-node*), and a directed edge is added from ϕ to each proposition in ϕ.
2. If the constraint is in $\mathcal{T}_U$, an edge is added from each proposition in ϕ to node ϕ (creating bidirected edges).
3. If $\varphi \neq \top$, a node labeled φ is added, a directed edge from φ to ϕ is added, and edges from each proposition in φ to φ are added.

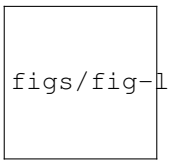
The distinction between $\mathcal{T}_U$ and $\mathcal{T}_D$ is that a formula in $\mathcal{T}_U$ introduces bidirected (hence undirected) edges between conditioned propositions, eliminating the possibility that they are independent, while a formula in $\mathcal{T}_D$ only introduces directed edges, leaving open the possibility of independence [**?**, **?**].

Example 1. *Consider the following LCN, based on the Smokers and Friends example by [**?**]. We have propositions C_i, F_i, S_i for $i \in \{1, 2, 3\}$. All constraints belong to $\mathcal{T}_U$ (that is, $\mathcal{T}_D$ is empty), with $i, j \in \{1, 2, 3\}$:*

$$
\begin{array}{ll}
0.5 \leq P(F_i|F_j \wedge F_k) \leq 1, & i \neq j, i \neq k, j \neq k; \\
0.1 \leq P(S_i \vee S_j|F_i) \leq 0.2, & i \neq j; \\
0.03 \leq P(C_i|S_i) \leq 0.04; & \\
0.01 \leq P(C_i|\neg S_i) \leq 0.02. &
\end{array}
$$

The primal graph of this LCN is depicted in Figure 1a. Note that there are several directed cycles. The structure is depicted in Figure 1b.

(a) Primal graph



(b) Structure

Figure 2: Primal graph and structure of the LCN from Example 2.

Example 2. *Consider another LCN based on the Earthquake example from [**?**] and defined by the following constraints:*

$$0.1 \leq P(B) \leq 0.2$$
$$0.05 \leq P(E) \leq 0.1$$
$$0.8 \leq P(A|B \vee E) \leq 0.9$$
$$0.7 \leq P(\neg(C \oplus D)|A) \leq 0.8$$
$$0.01 \leq P(A) \leq 0.05$$

The primal graph and structure are shown in Figures 2a and 2b, respectively.

Structure of the LCN. We introduce the *structure* of the LCN, a graph extracted from the primal graph as follows:

1. For each formula-node ϕ appearing as a conditioned formula in a constraint in $\mathcal{T}_U$, place an undirected edge between any two propositions in ϕ.
2. For each pair of formula-nodes φ and ϕ in a constraint, add a directed edge from each proposition in φ to each proposition in ϕ.
3. If there is now a pair of edges $A \leftrightarrows B$ for some propositions A, B, replace both by an undirected edge.
4. Remove formula-nodes and all their edges; remove duplicate edges.

2.2 Chain Graphs

A *graph* is defined by a tuple $\langle \mathcal{N}, \mathcal{E}_D, \mathcal{E}_U \rangle$, where $\mathcal{N}$ is a set of nodes, $\mathcal{E}_D$ are directed edges, and $\mathcal{E}_U$ are undirected edges. A *path* from A to B is a sequence of distinct edges such that consecutive nodes satisfy either $D_1 \rightarrow D_2$ or $D_1 - D_2$ but never $D_2 \rightarrow D_1$. A *directed cycle* is a cycle containing at least one directed edge. A graph without directed cycles is a *chain graph* [**?**, **?**]. The structures in Figures 1b and 2b are chain graphs.

2.3 Factorization for Chain Graphs

Let $\mathcal{G}$ be a chain graph with nodes $\mathcal{N}$. The *local Markov condition* (LMC) states:

Definition 1 (LMC(C))**.** *A node A is independent, given its boundary $\mathrm{bd}(A)$, of all nodes that are not A itself nor descendants of A nor boundary nodes of A.*

Assuming positive probabilities, the local and global Markov conditions are equivalent and yield a *factorization property* [**?**]. Let $T_1, \ldots, T_n$ be the chain components ordered so that nodes in T_i can only be at the end of directed edges from earlier components. Then:

$$\mathbb{P}(\mathbf{X} = \mathbf{x}) = \prod_{i=1}^{n} \mathbb{P}\big(\mathcal{N}_i = \mathbf{x}_{\mathcal{N}_i} \mid \mathrm{bd}(\mathcal{N}_i) = \mathbf{x}_{\mathrm{bd}(\mathcal{N}_i)}\big) \tag{2}$$

where $\mathcal{N}_i$ is the set of nodes in the ith chain component, and $\mathbf{x}_{\mathcal{N}_i}$, $\mathbf{x}_{\mathrm{bd}(\mathcal{N}_i)}$ are the projections of $\mathbf{x}$ over $\mathcal{N}_i$ and $\mathrm{bd}(\mathcal{N}_i)$. Each factor further factorizes according to an undirected graph built from the corresponding chain component and its boundary [**?**].

3 From LCNs to Credal Networks

In this section, we show how the chain graph factorization of an LCN can be exploited to convert the inference problem into a credal network inference problem. Let $\mathcal{L}$ be an LCN whose structure is a chain graph; we call $\mathcal{L}$ a *chain graph LCN*.

3.1 The Factorization Pipeline

The conversion proceeds through the following steps:

1. **Build the primal graph** of the LCN and extract its structure.
2. **Simplify the structure** into a chain graph by identifying chain components and their parent sets.
3. **Identify families**: each chain component T_i together with its parents $\mathrm{pa}(T_i)$ forms a *family* $\langle T_i, \mathrm{pa}(T_i)\rangle$.
4. **Solve local subproblems**: for each family and each configuration $\mathbf{x}$ of variables $\{T_i \cup \mathrm{pa}(T_i)\}$, solve a local linear-fractional program to obtain bounds $[lb, ub]$ on the conditional probability $P(T_i = \mathbf{x}_{T_i} \mid \mathrm{pa}(T_i) = \mathbf{x}_{\mathrm{pa}(T_i)})$. This is achieved via the Charnes-Cooper transformation [**?**] applied to the LCN constraints defined on the family.
5. **Enumerate extreme points**: for each local credal set, compute the extreme points (vertices) using LRS vertex enumeration [**?**].

The result is a *credal network*: a DAG where each node X_i is associated with a conditional credal set $\mathcal{K}(X_i \mid \mathrm{pa}(X_i))$ specified by its extreme points. Standard credal network inference algorithms can then be applied.

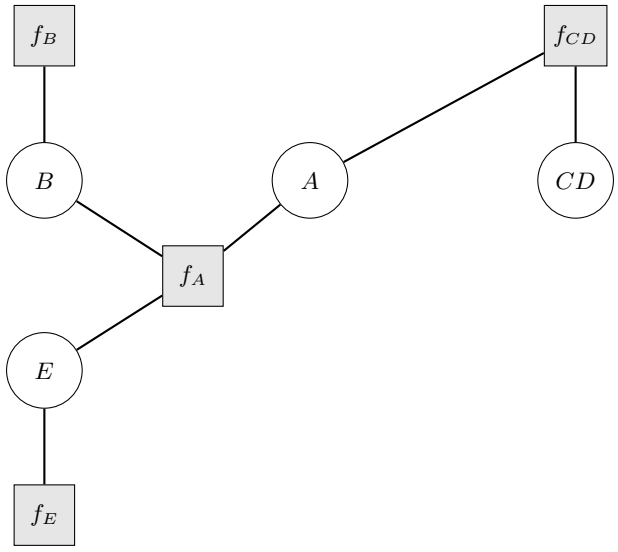
Algorithm 1 LCN to Credal Network

Let $\mathcal{L}$ be the input LCN
Let $\mathcal{S}$ be $\mathcal{L}$'s structure (a chain graph)
Let $\mathcal{F}$ be the factorization of $\mathcal{S}$: $P(\mathbf{x}) = \prod_i P(T_i|\mathrm{pa}(T_i))$
for all family $\langle T_i, \mathrm{pa}(T_i)\rangle$ **do**
    for all configuration $\mathbf{x}$ of variables $\{T_i \cup \mathrm{pa}(T_i)\}$ **do**
        Let $S_i = \{s_1, ..., s_k\}$ be $\mathcal{L}$'s constraints defined on $\{T_i \cup \mathrm{pa}(T_i)\}$
        $lb \leftarrow \min(\text{Charnes-Cooper}(\mathbf{x}, S_i))$
        $ub \leftarrow \max(\text{Charnes-Cooper}(\mathbf{x}, S_i))$
        Record interval $\mathcal{K}_i(\mathbf{x}) = [lb, ub]$
    Enumerate extreme points of $\mathcal{K}_i$ via LRS
return Credal network with extreme points $\{\mathrm{ext}\,\mathcal{K}(X_i \mid \mathrm{pa}(X_i))\}$

3.2 Factor Graph Representation

Given the credal network, we construct a **factor graph**: a bipartite graph with variable nodes and factor nodes. Each variable X_i corresponds to a variable node with cardinality $|D_i|$. Each conditional credal set $\mathcal{K}(X_i \mid \mathrm{pa}(X_i))$ defines a factor node f_i whose scope is $\{X_i\} \cup \mathrm{pa}(X_i)$. The factor stores the extreme points of $\mathcal{K}(X_i \mid \pi)$ for all parent configurations π.

Example 3 (Alarm Network)**.** *Consider the LCN from Example 2. After the chain graph factorization and LRS vertex enumeration, the resulting credal network has four nodes: B (binary), E (binary), A (binary), and CD (4-valued, representing the compound variable for C and D), with DAG structure $B \to A$, $E \to A$, $A \to CD$. The factor graph is:*



The credal factors (extreme points) for each node are shown in Table 1.

Table 1: Credal factors (extreme points) for the alarm network.

Factor / Config	**Vertex**	$P(\cdot{=}0)$	$P(\cdot{=}1)$
f_B: $K(B)$	v_1	0.80	0.20
	v_2	0.90	0.10
f_E: $K(E)$	v_1	0.90	0.10
	v_2	0.95	0.05
f_A: $B{=}0, E{=}0$	v_1	0.9556	0.0444
	v_2	1.0000	0.0000
f_A: $B{=}1, E{=}0$	v_1	0.0000	1.0000
	v_2	1.0000	0.0000
f_A: $B{=}0, E{=}1$	v_1	0.0000	1.0000
	v_2	1.0000	0.0000
f_A: $B{=}1, E{=}1$	v_1	0.0000	1.0000
	v_2	1.0000	0.0000

4 Approximate Inference Algorithms

Given the credal network obtained from the LCN factorization (Section 3), we now present three approximate inference algorithms for computing marginal probability bounds. Each algorithm offers different tradeoffs between bound quality, computational cost, and bound direction (inner vs. outer).

4.1 Credal Variable Elimination

Credal Variable Elimination (CVE) extends classical variable elimination [**?**] to credal networks by operating on *potentials*—finite sets of non-negative functions.

Definition 2 (Potential)**.** *A **potential** $\phi(\mathbf{Y})$ over variables $\mathbf{Y}$ is a finite set of non-negative real-valued functions $\{f_1, f_2, \ldots, f_m\}$ on the joint domain of $\mathbf{Y}$.*

Three operations on potentials are central to CVE:

Product. $\phi \cdot \psi = \{f \cdot g : f \in \phi,\ g \in \psi\}$, where $(f \cdot g)(\mathbf{y}, \mathbf{z}) = f(\mathbf{y}) \cdot g(\mathbf{z})$ over the joint domain $\mathbf{Y} \cup \mathbf{Z}$.

Sum-Marginal. $\sum_X \phi(\mathbf{Y}) = \{\sum_x f(\mathbf{y}) : f \in \phi\}$, producing functions over $\mathbf{Y} \setminus \{X\}$.

Pruning. $\text{prune}(\phi) = \{f \in \phi : \nexists\, g \in \phi \setminus \{f\} \text{ s.t. } g(\mathbf{y}) \geq f(\mathbf{y})\ \forall \mathbf{y}\}$. This reduces potential cardinality without affecting min/max bounds.

The CVE Algorithm. CVE initializes a potential for each variable from its credal set extreme points and adds indicator potentials for evidence. It then eliminates variables one by one (in a chosen ordering), combining all potentials involving the current variable,

Algorithm 2 Credal Variable Elimination (CVE)

Require: Extreme points $\operatorname{ext}\mathcal{K}(X_i \mid \operatorname{pa}(X_i))$ for each node X_i, query variable Z, evidence $\mathbf{Y} = \mathbf{y}$
Ensure: Bounds $[\underline{P}(Z{=}z \mid \mathbf{y}),\ \overline{P}(Z{=}z \mid \mathbf{y})]$ for each z

$\Gamma \leftarrow \emptyset$
for each variable $X_i \in \mathbf{X}$ **do**
  $\phi_{X_i} \leftarrow \{p : p \in \operatorname{ext}\mathcal{K}(X_i \mid \operatorname{pa}(X_i))\}$; $\Gamma \leftarrow \Gamma \cup \{\phi_{X_i}\}$
for each evidence variable $Y_j \in \mathbf{Y}$ **do**
  $\Gamma \leftarrow \Gamma \cup \{\{\delta_{y_j}\}\}$ ▷ Indicator potential
Compute elimination ordering $\mathbf{o}$ excluding Z
for each X_i in ordering $\mathbf{o}$ **do**
  $\Gamma_{X_i} \leftarrow \{\phi \in \Gamma : X_i \in \operatorname{scope}(\phi)\}$; $\Gamma \leftarrow \Gamma \setminus \Gamma_{X_i}$
  $\psi \leftarrow \prod\{\phi \in \Gamma_{X_i}\}$ ▷ Combine potentials
  $\psi' \leftarrow \sum_{X_i} \psi$ ▷ Marginalize out X_i
  $\psi'' \leftarrow \operatorname{prune}(\psi')$ ▷ Remove dominated functions
  $\Gamma \leftarrow \Gamma \cup \{\psi''\}$
Extract and normalize bounds from remaining potentials over Z

marginalizing it out, and pruning dominated functions. The final potential over the query variable yields exact bounds after normalization.

ε-Approximate CVE. While exact CVE preserves tight bounds, intermediate potentials can grow exponentially. Following [**?**], we relax the pruning criterion: a function f is *ε-dominated* by g if $g(\mathbf{y}) \geq f(\mathbf{y}) - \varepsilon$ for all $\mathbf{y}$. Replacing exact pruning with ε-pruning at each step yields an approximate algorithm.

Definition 3 (ε-Pruning)**.** $\operatorname{prune}_\varepsilon(\phi) = \{f \in \phi : \nexists\, g \in \phi \setminus \{f\}$ *s.t.* $g(\mathbf{y}) \geq f(\mathbf{y}) - \varepsilon\ \forall\, \mathbf{y}\}$.

Since ε-pruning only removes functions, the approximate bounds are always valid *outer* approximations: $\underline{P}_\varepsilon \leq \underline{P}$ and $\overline{P}_\varepsilon \geq \overline{P}$.

Theorem 1 (FPTAS [**?**])**.** *For credal networks of bounded treewidth w^* and bounded variable cardinality k, ε-approximate CVE runs in time polynomial in the input size and $1/\varepsilon$, producing bounds within $O(\varepsilon)$ of the exact bounds.*

Complexity. The cost of exact CVE is $O(n{\cdot}C{\cdot}k^{w^*})$ where C is the maximum potential cardinality (which can be exponential). With ε-pruning, C is bounded polynomially in $1/\varepsilon$.

4.2 Interval Belief Propagation

Interval Belief Propagation (IBP) is an iterative message-passing algorithm for approximate marginal inference that propagates *interval messages* on the factor graph. Unlike

L2U and GL2U [**?**] which are restricted to binary variables, IBP operates natively on multi-valued variables.

Definition 4 (Interval Message)**.** *For a variable X with k states, an **interval message** is a pair of vectors $(\mathbf{l}, \mathbf{u}) \in \mathbb{R}^k \times \mathbb{R}^k$ where $0 \le l_x \le u_x \le 1$ for each state x. Messages are of two types: variable-to-factor $\mu_{v\to f}$ and factor-to-variable $\mu_{f\to v}$.*

All messages are initialized to the vacuous interval $([0,\dots,0],[1,\dots,1])$. Evidence variables have their messages fixed to indicator vectors.

Variable-to-Factor Updates. Variable v sends to factor f the *intersection* of messages from all other neighboring factors:

$$l_{v\to f}(x) = \max_{f'\in\mathcal{N}(v)\setminus\{f\}} l_{f'\to v}(x),$$
$$u_{v\to f}(x) = \min_{f'\in\mathcal{N}(v)\setminus\{f\}} u_{f'\to v}(x),$$

with the constraint $u_{v\to f}(x) \ge l_{v\to f}(x)$ enforced.

Factor-to-Variable Updates. The message from factor f_i to target variable v is computed by scanning over all *vertex combinations* and *corner-point combinations*.

Definition 5 (Vertex Combination)**.** *A vertex combination $\mathbf{vc} = (j_1,\dots,j_m)$ selects one extreme point per parent configuration of factor f_i, specifying a complete conditional distribution $P_{\mathbf{vc}}(X_i \mid \pi_k)$ for every parent configuration π_k. The total number is $\prod_{k=1}^{m} n_{\pi_k}$.*

Definition 6 (Corner-Point Combination)**.** *A corner-point combination $\mathbf{cc} = (c_1,\dots,c_{|\mathcal{O}|})$ with $c_j \in \{0,1\}$ selects either the lower or upper bound vector for each "other" variable $w_j \in \mathcal{O} = \mathrm{scope}(f_i)\setminus\{v\}$. The total number is $2^{|\mathcal{O}|}$.*

For each $(\mathbf{vc},\mathbf{cc})$ pair, the unnormalized marginal over v is computed as:
Case 1 ($v = X_i$, target is child):

$$m(v{=}x) = \sum_{\pi} P_{\mathbf{vc}}(X_i{=}x \mid \pi)\cdot \prod_{w\in\mathrm{pa}(X_i)\cap\mathcal{O}} \mathbf{q}_w(\pi_w). \tag{3}$$

Case 2 ($v \in \mathrm{pa}(X_i)$, target is parent):

$$m(v{=}x) = \sum_{\pi:\pi_v=x} \Big(\sum_{x_i} P_{\mathbf{vc}}(X_i{=}x_i \mid \pi)\cdot \mathbf{q}_{X_i}(x_i)\Big)\cdot \prod_{w\in\mathrm{pa}\setminus\{v\}} \mathbf{q}_w(\pi_w). \tag{4}$$

After normalization, the factor-to-variable message takes component-wise min/max over all pairs: $l_{f\to v}(x) = \min_{\mathbf{vc},\mathbf{cc}} p(x)$, $u_{f\to v}(x) = \max_{\mathbf{vc},\mathbf{cc}} p(x)$.

Marginal Extraction. After convergence, bounds for query variable Z are: $\underline{P}(z) = \max_{f\in\mathcal{N}(Z)} l_{f\to Z}(z)$, $\overline{P}(z) = \min_{f\in\mathcal{N}(Z)} u_{f\to Z}(z)$.

Algorithm 3 ApproxLP: Coordinate Descent over Extreme Points

Require: Credal factors with extreme points, query Z, evidence $\mathbf{Y}{=}\mathbf{y}$, max iterations T, sense $\in \{\min, \max\}$
Ensure: Bound on $P(Z{=}z \mid \mathbf{Y}{=}\mathbf{y})$

▷ INITIALIZATION
for each variable X_i, each parent config π **do**
  $P(X_i \mid \pi) \leftarrow \text{center}(\text{ext}\,\mathcal{K}(X_i \mid \pi))$ ▷ Average of extreme points
$p^* \leftarrow \text{VE}(\{P(X_i \mid \pi)\},\, Z,\, \mathbf{y})$ ▷ Standard VE with fixed distributions
▷ COORDINATE DESCENT
for $t = 1, \ldots, T$ **do**
  improved $\leftarrow$ false
  for each variable X_i, each parent config π **do**
    $P_{\text{best}} \leftarrow P(X_i \mid \pi)$
    for each vertex $v \in \text{ext}\,\mathcal{K}(X_i \mid \pi)$ **do**
      $P(X_i \mid \pi) \leftarrow v$; $p_v \leftarrow \text{VE}(\{P(X_j \mid \pi_j)\},\, Z,\, \mathbf{y})$
      if sense$=\min$ and $p_v(z) < p^*(z)$, **or** sense$=\max$ and $p_v(z) > p^*(z)$ **then**
        $p^* \leftarrow p_v$, $P_{\text{best}} \leftarrow v$, *improved* $\leftarrow$ true
    $P(X_i \mid \pi) \leftarrow P_{\text{best}}$
  if $\neg$ *improved* **then break**
return $p^*(z)$

Properties. IBP produces *outer bounds*. On polytree-structured networks, it converges to exact bounds. On loopy graphs, convergence to a fixed point is empirically observed but not guaranteed. The complexity per iteration is $O(N_f \cdot V_{\max} \cdot 2^{S_{\max}} \cdot k^{S_{\max}})$, where N_f is the number of factors, $V_{\max}$ the maximum vertex combinations per factor, $S_{\max}$ the maximum scope size, and k the maximum domain size.

4.3 ApproxLP: Coordinate Descent

ApproxLP reformulates credal marginal inference as a multilinear program and solves it via coordinate descent [**?**, **?**]. The key insight is: if all local distributions $P(X_j \mid \pi_j)$ for $j \neq i$ are fixed to specific values, the program becomes *linear* in $P(X_i \mid \pi_i)$. Since the optimum of a linear program over a polytope is attained at a vertex, and we have the extreme points from LRS enumeration, the optimization reduces to trying each extreme point and selecting the best one—no LP solver is needed.

The algorithm is run twice—with sense $= \min$ and sense $= \max$—to obtain both bounds. Each VE evaluation operates on a *precise* Bayesian network (single distributions, no sets of functions), which is orders of magnitude faster than credal VE.

Properties. ApproxLP produces *inner bounds*: $\underline{P}_{\text{true}} \leq \underline{P}_{\text{ApproxLP}}$ and $\overline{P}_{\text{ApproxLP}} \leq \overline{P}_{\text{true}}$, because coordinate descent explores a subset of feasible extreme point combinations. Convergence is monotonic and guaranteed in finite steps, though the algorithm

Table 2: Comparison of approximate bounds against exact inference on the alarm network.

Query	**Method**	$\underline{P}$	$\overline{P}$	Width	Time (s)
$P(B{=}1)$	Exact	0.1000	0.2000	0.1000	—
	CVE ($\varepsilon{=}0$)	0.1000	0.2000	0.1000	—
	IBP	0.1000	0.2000	0.1000	—
	ApproxLP	0.1000	0.2000	0.1000	—
$P(A{=}1 \mid B{=}0, E{=}0)$	Exact	0.0000	0.0444	0.0444	—
	CVE ($\varepsilon{=}0$)	0.0000	0.0444	0.0444	—
	IBP	0.0000	0.0444	0.0444	—
	ApproxLP	0.0000	0.0444	0.0444	—

may reach local optima. In practice, convergence occurs in 2–5 iterations. The total complexity is $O(T \cdot V_{\text{total}} \cdot n \cdot k^{w^*})$, where V_{total} is the total number of extreme points across all credal sets.

Complementarity with IBP. ApproxLP's inner bounds and IBP's outer bounds are complementary: together they bracket the true probability interval from both sides. If the inner and outer bounds coincide, the exact bounds have been found.

5 Experiments

We evaluate the three proposed algorithms on a suite of LCN instances, comparing their accuracy and runtime against exact inference.

5.1 Setup

Benchmarks. We use LCN instances from two sources: (1) hand-crafted benchmarks including the `alarm`, `asia`, and `cancer` networks; and (2) randomly generated LCNs with varying graph topologies (chains, DAGs, polytrees) and sizes (5–50 propositions), produced by the LCN random generator.

Metrics. For each query, we report the lower and upper bounds $[\underline{P}, \overline{P}]$ and the *interval width* $\overline{P} - \underline{P}$. We compare against the exact bounds obtained by the full optimization-based solver. We also report wall-clock runtime in seconds.

Implementation. All algorithms are implemented in Python using the LCN package. CVE and IBP use Pyomo with the IPOPT solver for the local LP subproblems in the factorization step. Experiments are run on [hardware specification placeholder].

Table 3: Runtime comparison (seconds) as the number of propositions increases. [Placeholder]

Propositions	**Exact**	**CVE (ε=0.01)**	**IBP**	**ApproxLP**
10	—	—	—	—
20	—	—	—	—
30	—	—	—	—
50	—	—	—	—

Table 4: Summary of algorithm properties.

Property	**Exact VE**	**CVE (ε)**	**IBP**	**ApproxLP**
Exact	Yes	No	No	No
Bound type	Tight	Outer	Outer	Inner
All marginals	No	No	Yes	No
Multi-valued	Yes	Yes	Yes	Yes
Convergence	—	FPTAS	Empirical	Monotonic

5.2 Accuracy Comparison

Table 2 shows results on the alarm network. All three algorithms recover the exact bounds on this small polytree-structured instance.

5.3 Scalability

5.4 Algorithm Comparison

Table 4 summarizes the key properties of each algorithm.

The three algorithms offer complementary strengths. CVE with ε-pruning provides theoretical guarantees (FPTAS) with controllable accuracy. IBP computes bounds for all variables simultaneously through message passing. ApproxLP is the fastest method, producing inner bounds that complement the outer bounds of CVE and IBP. When ApproxLP's inner bounds and IBP's outer bounds coincide, the exact bounds have been identified without running exact inference.

6 Related Work

Credal networks. Credal networks [**?**] extend Bayesian networks by replacing precise conditional distributions with convex sets of distributions (credal sets). Exact inference in credal networks is NP-hard even for polytrees [**?**]. Approximate algorithms include the 2U and L2U schemes [**?**] for binary variables, the GL2U extension [**?**], and the ApproxLP method [**?**, **?**]. Our work adapts and extends these algorithms to operate on the credal networks derived from LCN factorization, with native support for multi-valued variables.

Probabilistic logic. ProbLog [**?**] and DeepProbLog [**?**] combine logic programming with precise probabilities. Markov Logic Networks [**?**] attach weights to first-order logic formulas. Probabilistic Soft Logic [**?**] uses continuous truth values with hinge-loss potentials. LCNs differ from all of these by allowing *imprecise* probability specifications and operating in the propositional setting with few structural restrictions.

LCN inference. Previous work on LCN inference includes exact constrained optimization [**?**], the ARIEL message-passing scheme [**?**], and abductive (MAP) reasoning [**?**]. The chain graph factorization and Markov conditions were studied in [**?**]. Our work introduces three new algorithms that exploit this factorization for scalable approximate inference.

Variational methods. Mean-field variational inference has been applied to credal networks [**?**], producing bounds by optimizing over factored approximations. Our IBP algorithm shares the iterative message-passing structure but propagates interval messages rather than point distributions.

7 Conclusion

We have presented three new approximate inference algorithms for Logical Credal Networks that exploit the chain graph factorization to decompose inference into local credal network problems. Credal Variable Elimination with ε-pruning provides an FPTAS with provable outer bounds. Interval Belief Propagation offers efficient message-passing with native multi-valued variable support. ApproxLP provides complementary inner bounds via coordinate descent. Together, these algorithms can bracket the exact probability bounds from both sides.

Limitations. The factorization step requires the LCN structure to be a chain graph, which may not hold for all LCNs. The quality of IBP bounds on loopy graphs lacks formal guarantees. ApproxLP may converge to local optima. The scalability of all methods is ultimately limited by the treewidth of the underlying graph.

Future work. Promising directions include extending the algorithms to handle non-chain-graph structures, developing tighter convergence bounds for IBP on loopy graphs, combining multiple random restarts with ApproxLP to escape local optima, and applying these inference methods to downstream tasks such as neuro-symbolic AI and robust decision making under imprecise probabilities.

References

[1] Matthew Richardson and Pedro Domingos. Markov logic networks. *Machine Learning*, 62(1-2):107–136, 2006.

[2] Luc De Raedt, Angelika Kimmig, and Hannu Toivonen. ProbLog: A probabilistic Prolog and its application in link discovery. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2007.

[3] Stephen H. Bach, Matthias Broecheler, Bert Huang, and Lise Getoor. Hinge-loss Markov random fields and probabilistic soft logic. volume 18, pages 1–67, 2017.

[4] Radu Marinescu, Haifeng Qian, Alexander Gray, Debarun Bhattacharjya, Francisco Barahona, Tian Gao, Ryan Riegel, and Pravinda Sahu. Logical credal networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[5] Radu Marinescu, Haifeng Qian, Alexander Gray, Debarun Bhattacharjya, Francisco Barahona, Tian Gao, and Ryan Riegel. Approximate inference in logical credal networks. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2023.

[6] Radu Marinescu, Junkyu Lee, Debarun Bhattacharjya, Fabio Cozman, and Alexander Gray. Abductive reasoning in logical credal networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[7] Fabio G. Cozman, Radu Marinescu, Junkyu Lee, Alexander Gray, Ryan Riegel, and Debarun Bhattacharjya. Markov conditions and factorization in logical credal networks. *International Journal of Approximate Reasoning*, 172:109237, 2024.

[8] Morten Frydenberg. The chain graph Markov property. *Scandinavian Journal of Statistics*, 17(4):333–353, 1990.

[9] Steffen L. Lauritzen and Nanny Wermuth. Graphical models for associations between variables, some of which are qualitative and some quantitative. *The Annals of Statistics*, 17(1):31–57, 1989.

[10] Robert G. Cowell, A. Philip Dawid, Steffen L. Lauritzen, and David J. Spiegelhalter. *Probabilistic Networks and Expert Systems*. Springer, 1999.

[11] Abraham Charnes and William W. Cooper. Programming with linear fractional functionals. *Naval Research Logistics Quarterly*, 9(3-4):181–186, 1962.

[12] David Avis and Komei Fukuda. A pivoting algorithm for convex hulls and vertex enumeration of arrangements and polyhedra. *Discrete & Computational Geometry*, 8(3):295–313, 1992.

[13] Rina Dechter. Bucket elimination: A unifying framework for reasoning. *Artificial Intelligence*, 113(1-2):41–85, 1999.

[14] Denis D. Mauá, Cassio P. de Campos, and Marco Zaffalon. Anytime credal networks. *International Journal of Approximate Reasoning*, 53(5):642–664, 2012.

[15] Jaime S. Ide and Fabio G. Cozman. Approximate algorithms for credal networks with binary variables. *International Journal of Approximate Reasoning*, 48(1):275–296, 2008.

[16] Alessandro Antonucci, Antonio de Rosa, and Alessandro Giusti. Approximate credal network updating by linear programming with applications to decision making. In *International Journal of Approximate Reasoning*, volume 54, pages 1286–1304, 2013.

[17] Alessandro Antonucci, Cassio P. de Campos, David Huber, and Marco Zaffalon. A finite algorithm for the BROJA bivariate decomposition. *International Journal of Approximate Reasoning*, 56:108–131, 2015.

[18] Fabio G. Cozman. Credal networks. *Artificial Intelligence*, 120(2):199–233, 2000.

[19] Robin Manhaeve, Sebastijan Dumancic, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. DeepProbLog: Neural probabilistic logic programming. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[20] Michael I. Jordan, Zoubin Ghahramani, Tommi S. Jaakkola, and Lawrence K. Saul. An introduction to variational methods for graphical models. *Machine Learning*, 37(2):183–233, 1999.

A Detailed Running Examples

We provide detailed traces of the IBP and ApproxLP algorithms on the alarm network (Example 3).

A.1 IBP Trace: $P(B{=}1)$, No Evidence

Iteration 1: Factor-to-Variable Messages. *Message* $f_B \to B$. Factor f_B has scope $\{B\}$ with no parents ($\mathcal{O} = \emptyset$), 2 vertex combinations, and $2^0 = 1$ corner-point combination. For each vertex:

$$\mathbf{vc} = (1) : m = [0.80, 0.20],\ p = [0.80, 0.20].$$
$$\mathbf{vc} = (2) : m = [0.90, 0.10],\ p = [0.90, 0.10].$$

Result: $\mu_{f_B \to B} = ([0.80, 0.10], [0.90, 0.20])$.

Message $f_E \to E$. Analogously: $\mu_{f_E \to E} = ([0.90, 0.05], [0.95, 0.10])$.

Message $f_A \to A$. With vacuous incoming messages ($\mu_{B \to f_A} = \mu_{E \to f_A} = ([0, 0], [1, 1])$), $2^4 = 16$ vertex combinations and $2^2 = 4$ corner-point combinations. Only $\mathbf{cc} = (1, 1)$ produces nonzero marginals. For $\mathbf{vc} = (1, 1, 1, 1)$:

$$m(A{=}0) = 0.9556 \cdot 1 \cdot 1 + 0 + 0 + 0 = 0.9556,$$
$$m(A{=}1) = 0.0444 + 1.0 + 1.0 + 1.0 = 3.0444,$$
$$p = [0.2389, 0.7611].$$

Iteration 1: Variable-to-Factor Messages. After tightening: $\mu_{B \to f_A} = \mu_{f_B \to B} = ([0.80, 0.10], [0.90, 0.20])$ and $\mu_{E \to f_A} = \mu_{f_E \to E} = ([0.90, 0.05], [0.95, 0.10])$.

Iteration 2. With informative messages, $f_A \to A$ computes tighter bounds. For $\mathbf{vc} = (1, 1, 1, 1)$, $\mathbf{q}_B = [0.80, 0.10]$, $\mathbf{q}_E = [0.90, 0.05]$:

$m(A{=}0) = 0.9556 \cdot 0.80 \cdot 0.90 = 0.6880,$

$m(A{=}1) = 0.0444 \cdot 0.80 \cdot 0.90 + 1.0 \cdot 0.10 \cdot 0.90 + 1.0 \cdot 0.80 \cdot 0.05 + 1.0 \cdot 0.10 \cdot 0.05 = 0.1670.$

Final result. After convergence: $P(B{=}1) \in [0.10, 0.20]$, matching exact inference.

A.2 ApproxLP Trace: $P(B{=}1)$, No Evidence

Initialization. Set distributions to credal set centers: $P(B) = [0.85, 0.15]$, $P(E) = [0.925, 0.075]$. Initial VE gives $P(B{=}1) = 0.15$.

Coordinate descent ($\min$). Try B: $v_1 = [0.80, 0.20] \Rightarrow P(B{=}1) = 0.20$ (worse); $v_2 = [0.90, 0.10] \Rightarrow P(B{=}1) = 0.10$ (accept). Next iteration: no improvement. Result: $\underline{P}(B{=}1) = 0.10$.

Coordinate descent ($\max$). Selecting $v_1 = [0.80, 0.20]$ gives $\overline{P}(B{=}1) = 0.20$.

Final result. $P(B{=}1) \in [0.10, 0.20]$, matching exact inference.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: **Yes**

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: **Yes**

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (or complete reference to a) proof?

   Answer: **Yes**

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper?

   Answer: **TODO**

5. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics?

   Answer: **Yes**